

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА
ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ
ІНЖЕНЕРІЇ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРА ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 3

виконала:
студентка групи КН-25-1
Бойко Д.О.
Перевірив:
Старший викладач кафедри АІС
Сидоренко В.М.

Кременчук 2026

Практична робота № 3

Тема. Алгоритми сортування та їх складність. Порівняння алгоритмів сортування

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Постановка завдання:

Виконати індивідуальне завдання.

Для практичної роботи №3 студенти виконують єдиний варіант та розв'язують усі задачі.

Завдання 1

Вивчити самостійно і записати (будь-яким способом) алгоритм бульбашкового сортування. Оцінити асимптотику алгоритму сортування методом бульбашки в найгіршому і в найкращому випадку. Порівняти за цими показниками бульбашковий алгоритм з алгоритмом сортування вставленням. Чому на практиці бульбашковий алгоритм виявляється менш ефективним у порівнянні з сортуванням методом зливанням?

Розв'язання:

Алгоритм бульбашкового сортування

Бульбашкове сортування працює шляхом багаторазового порівняння сусідніх елементів масиву та їх перестановки, якщо вони розташовані у неправильному порядку.

Бульбашковий код:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Найгірший випадок

Найгірший випадок виникає тоді, коли масив відсортований у зворотному порядку.

Кількість порівнянь:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$$

Отже: $O(n^2)$

Найкращий випадок

Якщо масив уже відсортований, алгоритм усе одно виконує перевірки.

Тому складність: $O(n^2)$

Порівняння з сортуванням вставлянням

Алгоритм	Найгірший випадок	Найкращий випадок
Бульбашкове сортування	$O(n^2)$	$O(n^2)$
Сортування вставлянням	$O(n^2)$	$O(n)$

Сортування вставлянням працює швидше для майже відсортованих масивів.

Чому бульбашкове сортування менш ефективно за сортування злиттям?

Сортування злиттям має складність:

$$O(n \log n)$$

тому для великих наборів даних воно працює значно швидше, ніж бульбашкове сортування з квадратичною складністю.

Завдання 2

Оцінити асимптотичну складність алгоритму сортування зливанням, скориставшись основною теоремою рекурсії.

Розв'язання

Алгоритм сортування злиттям ділить масив на дві частини, сортує їх рекурсивно та об'єднує результати.

Рекурентне співвідношення:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

де:

- $2T\left(\frac{n}{2}\right)$ — сортування двох половин;
- $O(n)$ — об'єднання масивів.

За основною теоремою рекурсії:

$$a = 2, b = 2, f(n) = O(n)$$

Обчислимо: $n^{\log_{b^a} n^{\log_{2^2}}} = n$

Отже:

$$f(n) = \Theta(n^{\log_{b^a}})$$

Тому:

$$T(n) = \Theta(n \log n)$$

Відповідь: $\Theta(n \log n)$

Завдання 3

Вивчити і записати (будь-яким способом) самостійно алгоритм швидкого сортування. Оцінити асимптотичну складність алгоритму швидкого сортування, скориставшись основною теоремою рекурсії.

Алгоритм сортування:

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
  
    return quick_sort(left) + middle + quick_sort(right)
```

Розв'язання

Швидке сортування працює за принципом «розділай і володарюй».

Обирається опорний елемент, після чого масив поділяється:

- елементи менші — ліворуч;
- більші — праворуч.

Середній випадок

Рекурентне співвідношення:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Тому: $O(n \log n)$

Найгірший випадок

Виникає тоді, коли опорний елемент щоразу найбільший або найменший.

Тоді:

$$T(n) = T(n - 1) + O(n)$$

Отже:

$$O(n^2)$$

Переваги швидкого сортування

- висока швидкість роботи;
- ефективність на практиці;
- не потребує додаткової великої пам'яті.

Отримані результати

У ході практичної роботи:

- розглянуто основні алгоритми сортування;
- визначено їх асимптотичну складність;
- виконано порівняння ефективності алгоритмів;
- досліджено переваги та недоліки різних методів сортування.

Контрольні питання

1. Що таке асимптотична складність алгоритму сортування і чому вона важлива?

Асимптотична складність показує, як змінюється час роботи алгоритму при збільшенні кількості даних. Вона дозволяє порівнювати ефективність алгоритмів.

2. Які алгоритми мають квадратичну складність?

Квадратичну складність мають:

- бульбашкове сортування;
- сортування вставленням;
- сортування вибором.

Для великих наборів даних вони працюють повільно.

3. У чому перевага сортування злиттям над сортуванням вставленням?

Сортування злиттям має складність $O(n \log n)$, тому краще підходить для великих масивів.

4. Які алгоритми використовуються у стандартних бібліотеках мов програмування?

- Python — Timsort;
- Java — QuickSort або MergeSort;
- C++ — IntroSort.

5. Яка різниця між MergeSort і QuickSort?

- MergeSort стабільний і гарантує $O(n \log n)$;

- QuickSort швидший на практиці, але може мати $O(n^2)$.

6. Які фактори враховують при виборі алгоритму сортування?

Враховують: розмір даних; необхідність стабільності; використання пам'яті; швидкість роботи; ступінь відсортованості масиву.